

~/typecode - splash

<typecode/>

Тренажёр слепой печати для программистов

Терминальное приложение на чистом C + ncurses

ПРОЕКТ

учебный проект

REPO

github.com/ginluv21/typecode

STACK

C · ncurses · GNU Make

// О проекте

Не просто тренажёр - инструмент для разработчика

description.txt

typecode - это **TUI-тренажёр** печати для программистов.

Обучает набору **настоящего кода**
- синтаксиса, паттернов,
конструкций реальных языков.

```
// ощущается как настоящая  
// Linux-утилита, не игрушка  
// в духе btop / lazygit / htop
```

\$ goals

- ▶ **Глубоко изучить C**
низкоуровневое
программирование
- ▶ **Сильный open-source**
проект в портфолио
- ▶ **Реально полезный**
инструмент,
которым пользуются

// Проблема

Почему обычные тренажёры не подходят программистам

✗ обычные тренажёры

- Тексты на естественном языке
- Никаких { } ; -> ::
- Привычка к словам, не к коду
- Не учат раскладке для IDE
- WPM на коде остаётся низким

✓ <typecode />

- Уроки на реальном коде
- C / Python / JS / Bash / Go
- Привычка к синтаксису
- Спец. символы из коробки
- Heatmap слабых мест

// Возможности

Что уже работает в v2.0

[01]

TUI-меню

Главное меню,
stats, settings,
resize терминала

[02]

120 уроков

20 latin
+ 20×5 языков
с прогрессией

[03]

5 языков

C / Python / JS
Bash / Go -
реальные сниппеты

[04]

Метрики

WPM, accuracy
ошибки + история
всех сессий

[05]

Режимы

Hardcore (no
Backspace),
Timed, Infinite

[06]

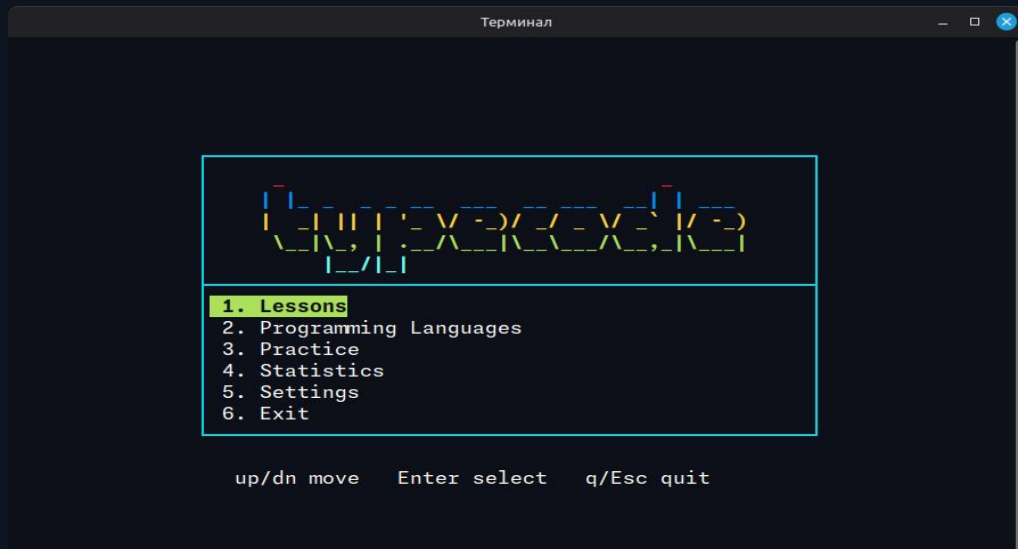
Heatmap ★

ASCII keyboard
+ weak spots
+ авто-тренировка

// Главное меню

ASCII-логотип + 6 пунктов навигации

~/typecode - main



navigation

Меню:

1. Lessons
2. Programming Langs
3. Practice
4. Statistics
5. Settings
6. Exit

Управление:

↑/↓ навигация
Enter выбор
q/Esc выход

// Уроки

120 уроков всего - пример Python

python lessons

```
Терминал
Lessons
01 basics
02 lists
03 functions
04 classes
05 dicts
06 tuples sets
07 comprehensions
08 exceptions
09 file io
10 generators
11 decorators
12 inheritance
13 itertools
14 dataclasses
15 sorting
16 functional
17 recursion
18 context managers
19 algorithms
up/dn - move  Enter - start  Esc - back
```

120 lessons

Всего уроков:

20 latin
20 C
20 Python
20 JavaScript
20 Bash
20 Go

120 файлов.txt

// добавление
// нового урока
// = новый .txt

// Сессия + Results

Цветовая подсветка + сводка после

10 generators

10 generators

```
def count_up(n):  
    for i in range(n):  
        yield i
```

```
gen = count_up(5)  
print(next(gen))
```

```
def squares():$  
> n = 0$  
> while True:$  
> > yield n * n$  
> > n += 1$
```

WPM: 18 Accuracy: 98% Errors: 2

Esc - finish early

results

10 generators

WPM	18
Accuracy	98%
Errors	2
Time	1:48

[R] Retry [Esc] Lessons [Q] Main menu

// Programming Languages

100 уроков на пяти языках

\$ language coverage

► **C** 20 уроков

variables → final

► **Python** 20 уроков

basics → algorithms

► **JavaScript** 20 уроков

ES6 + async

► **Bash** 20 уроков

scripts + pipes

► **Go** 20 уроков

goroutines + channels

≅ 100 уроков по реальному коду

go/02_functions.txt

```
func add(a int, b int) int {  
    return a + b  
}
```

```
result := add(2, 3)
```

```
// настоящий Go-код,  
// а не выдуманный текст
```

// Hardcore + Timed

Сложные режимы для прокачки

20 final [HARDCORE]

20 final

[HARDCORE]

```
int binary_search(int *arr, int n, int target) {  
    int left = 0;  
    int right = n - 1;$  
    while (left <= right) {$  
        > int mid = left + (right - left) / 2;$  
        > if (arr[mid] == target) return mid;$  
        > if (arr[mid] < target) left = mid + 1;$  
        > else right = mid - 1;$  
    }$  
    return -1;$  
}$
```

WPM: 18 Accuracy: 90% Errors: 8

[41s] Esc - finish early

modes

► HARDCORE

Backspace OFF.

Ошибки нельзя
исправить, курсор
идёт дальше.

► TIMED

Лимит 15-300 сек.

Обратный отсчёт
[41s] справа

► INFINITE

Зацикленный текст,
выход по Esc

// Settings + Statistics

Сохранение между сессиями

settings.conf

```
Терминал

Settings

Hardcore:    On
Mode:        Timed
Time limit:  90s

up/dn move  left/right change  Esc save & exit
```

stats.txt

```
Терминал

Statistics

Best WPM:      106
Avg WPM:       32 (last 10)
Avg accuracy:  66.4%
Total sessions: 18

Recent sessions:

Lesson          WPM  Accuracy  Errors
-----
10 generators    18    98%       2
09 enums         20   100%       0
01 home row      43    20%       8
01 variables     106  100%       0
01 home row      66    24%      71
04 loops         17   100%       0
01 variables     10     6%      93
01 home row      17   100%       0
01 home row      12    20%       4
Esc / Q - back  H - heatmap  W - weak spots  up/down - scroll
```

// Settings → ~/.typecode/settings.conf

Statistics → ~/.typecode/stats.txt

// Keyboard Heatmap

Главная фишка проекта • накапливается между всеми сессиями

Keyboard Heatmap

```
Терминал

Keyboard Heatmap

` 1 2 3 4 5 6 7 8 9 0 - =
q w e r t y u i o p [ ] \
a s d f g h j k l ; '
z x c v b n m , . /

Color: green < 3 err.  yellow < 10  red >= 10
R - reset  Esc - back
```

как работает

- < 3 ошибок
- < 10 ошибок
- ≥ 10 ошибок

Хранение:

~/.typecode/
heatmap.txt

// данные не
// сбрасываются

// Weak Spots - авто-тренировка

Программа сама делает упражнение под твои ошибки

Your weak spots

```
Терминал

Your weak spots

Top problem symbols:

1. space - 14 errors
2. j - 12 errors
3. f - 10 errors
4. d - 9 errors
5. k - 9 errors
6. a - 6 errors
7. l - 6 errors
8. s - 6 errors
9. ; - 4 errors
10. 10 - 3 errors

G - generate exercise
Esc - back
```

workflow

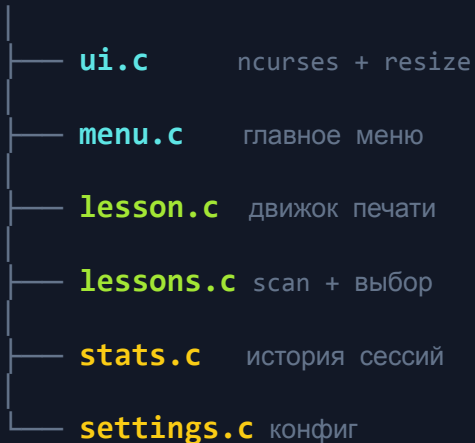
1. `typecode` копит
ошибки за
все сессии
2. топ-10 берётся
по weak spots
3. клавиша **G**
генерирует
упражнение
4. typing engine
→ **тренируешь**

// Архитектура

7 модулей, каждый = пара .c/.h с чёткой ответственностью

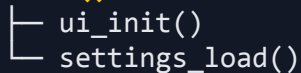
module graph

main.c



control flow

1. main()



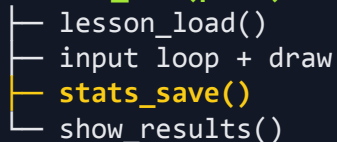
2. menu_run()

→ MenuOption (enum)

3. lessons_scan(dir)

└ список .txt

4. lesson_run(path)



// Структуры данных

Lesson, Metrics, CharState - ядро модуля lesson.c

include/lesson.h

```
typedef struct {  
    char *text;  
    int len;  
    char name[LESSON_NAME_MAX];  
} Lesson;
```

```
typedef struct {  
    int correct, errors, wpm;  
    float accuracy;  
    int duration_sec, started;  
    struct timespec start;  
} Metrics;
```

```
typedef enum { CHAR_UNTYPED, CHAR_CORRECT, CHAR_WRONG }  
CharState;
```

// объяснения

► Lesson

текст урока
+ имя файла

► Metrics

счётчики +
high-res таймер
(timespec)

► CharState

состояние
каждого символа
→ цвет на экране

// Метрика WPM

Как считается скорость в реальном времени

// формула

$$\text{WPM} = (\text{correct_chars} \div 5) \div \text{minutes_elapsed}$$

src/lesson.c

```
double minutes = duration_sec / 60.0;
metrics.wpm = (int)((metrics.correct /
                    5.0) / minutes);
```

// зачем ÷5

Стандарт WPM:

1 «СЛОВО» = 5 СИМВ.

Таймер: clock_gettime
(MONOTONIC) - точность
до наносекунд

// Стек технологий

Минимальный стек + 33 теста с покрытием 60%+

ЯЗЫК	TUI	СБОРКА	ТЕСТЫ	ЗАВИСИМ.
C (C11)	ncurses	GCC + Make	72 теста	libncurses
статическая типизация	терминальная графика	+ make coverage (gcovr)	8 файлов ≥ 95% покрытие	одна библиотека

✗ запрещено в проекте

no C++ · no GUI frameworks · no heavy libs · no over-engineering

// Roadmap

MVP / v1.0 / v2.0 - готово · v3.0+ - future work

MVP+v1.0

✓ DONE

- TUI меню
- Движок печати
- 20 latin + 100 lang
- 5 языков
- Practice / Load file
- Statistics + Settings
- Hardcore + Timed
- Resize терминала

v2.0

✓ DONE

- Heatmap клавиатуры
- Weak Spots
- Auto-генерация
- упражнений (G)
- 33 теста
- Coverage 60%+

v3.0+

○ FUTURE

- Кириллица (ncursesw)
- ✓ Vim mode (hjkl)
- ✓ Speed Challenge
- ранги S/A/B/C
- Splash screen
- Man page typecode.1

~/typecode - thanks

СПАСИБО ЗА ВНИМАНИЕ

team

- ▶ Антон Никита
- ▶ Ахмедов Бахтовар
- ▶ Габибуллаев Габибуллах
- ▶ Ефанов Семён

links

\$gitclone

github.com/ginluv21/typecode

\$ make && ./typecode

// вопросы?

// готовы показать живое демо